
Ignition IT/OT Lab

Portfolio · Christoph Viehoff, Automation and Controls Engineer

A working Ignition 8.3 lab built at home in Docker. It connects a simulated semiconductor etch tool over SECS/GEM, models dry vacuum pumps with a reusable UDT, and runs the operator screens in Perspective Workstation. Protocols, topic structure and tag design follow what a fab or plant uses.

CONTENTS

Page 2 **Lab screens**

Both demos running live in Perspective Workstation

Page 3 **SECS/GEM to Ignition**

Etch tool data up to SCADA over HSMS and MQTT, remote commands back down as S2F41

Page 6 **One Definition, Every Pump**

A VacuumPump UDT with limits, health logic, alarms and history, reused per pump

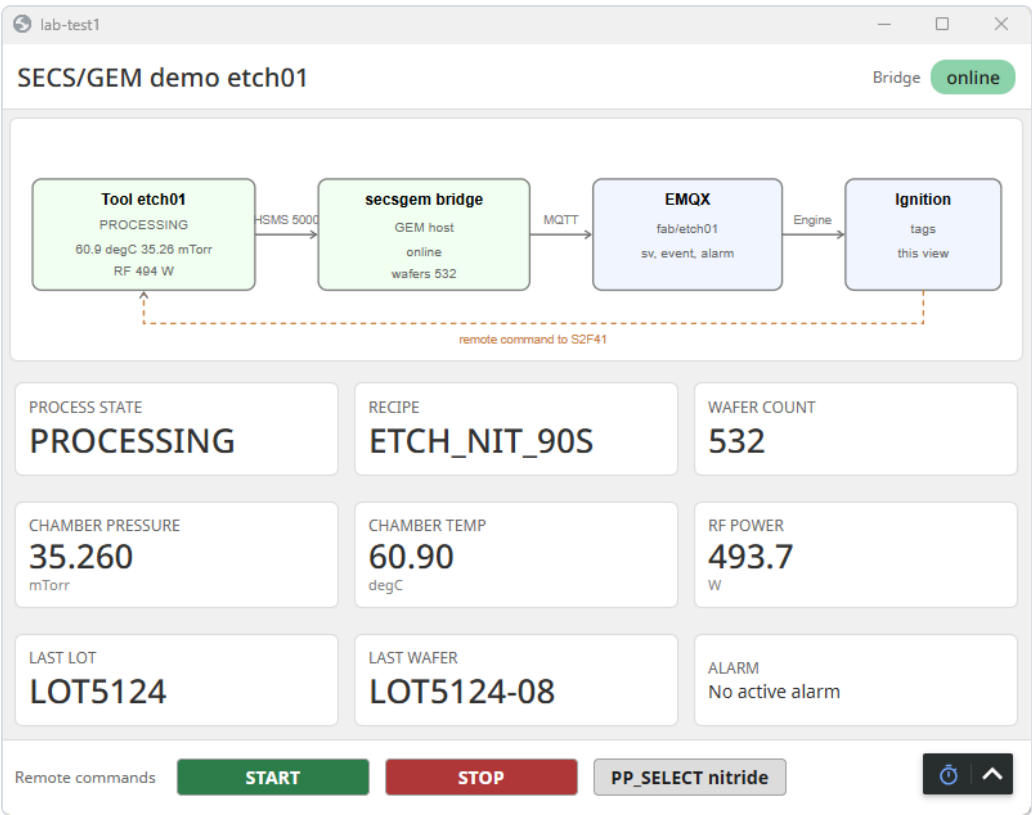
STACK

Ignition 8.3 (Perspective, UDTs, alarming, MQTT Engine) · SECS/GEM via Python secsgem · EMQX · PostgreSQL · OPC UA · Sparkplug B · Docker Compose · Git

SOURCE

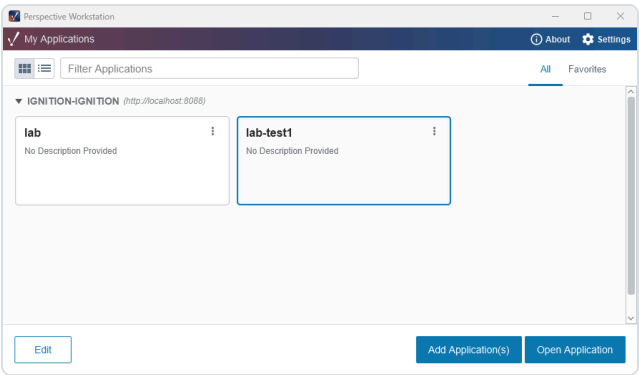
github.com/christophviehoff/ignition-lab

SECS/GEM etch tool



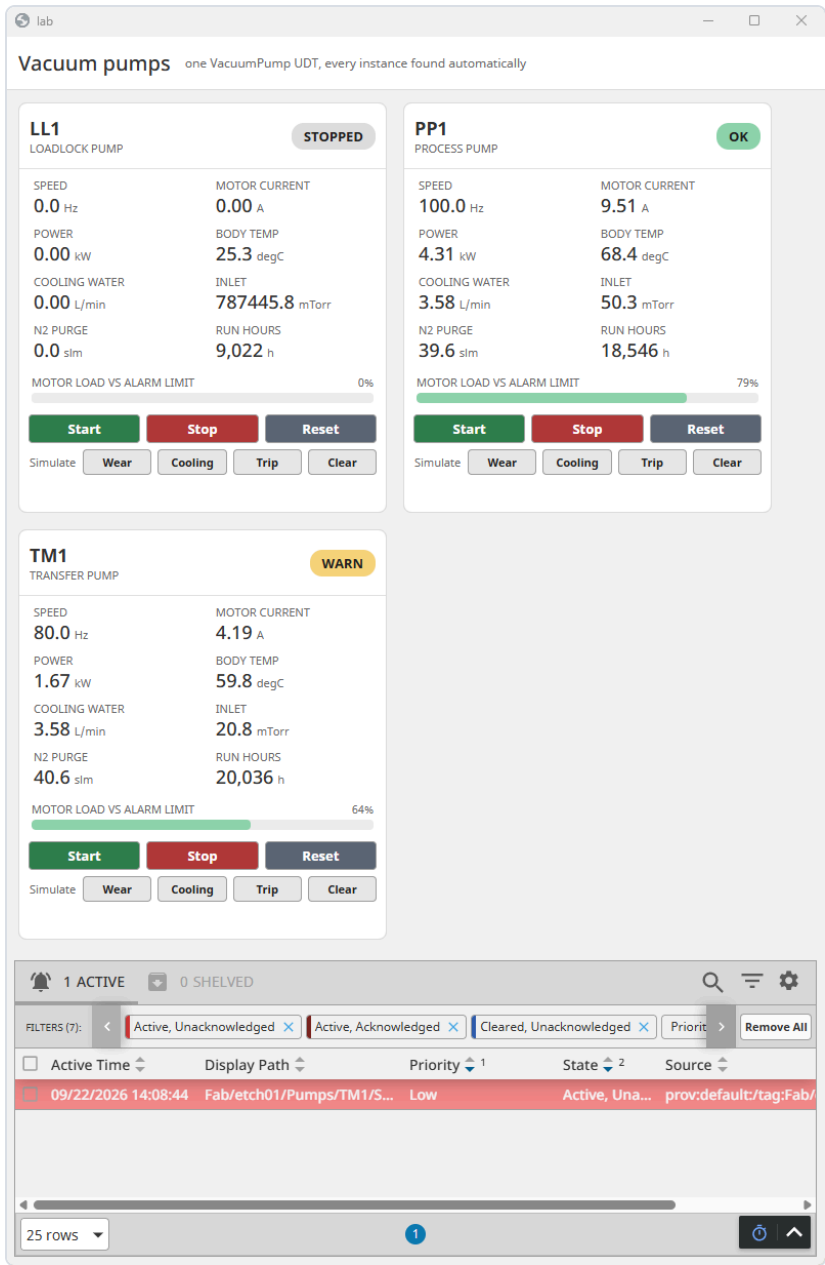
Live tool data over HSMS and MQTT. Buttons send remote commands back to the tool as S2F41.

Perspective Workstation



Both projects launched as desktop apps from the Docker gateway.

Vacuum pumps from one UDT



Three pump instances with health states and UDT alarms.

SECS/GEM to Ignition

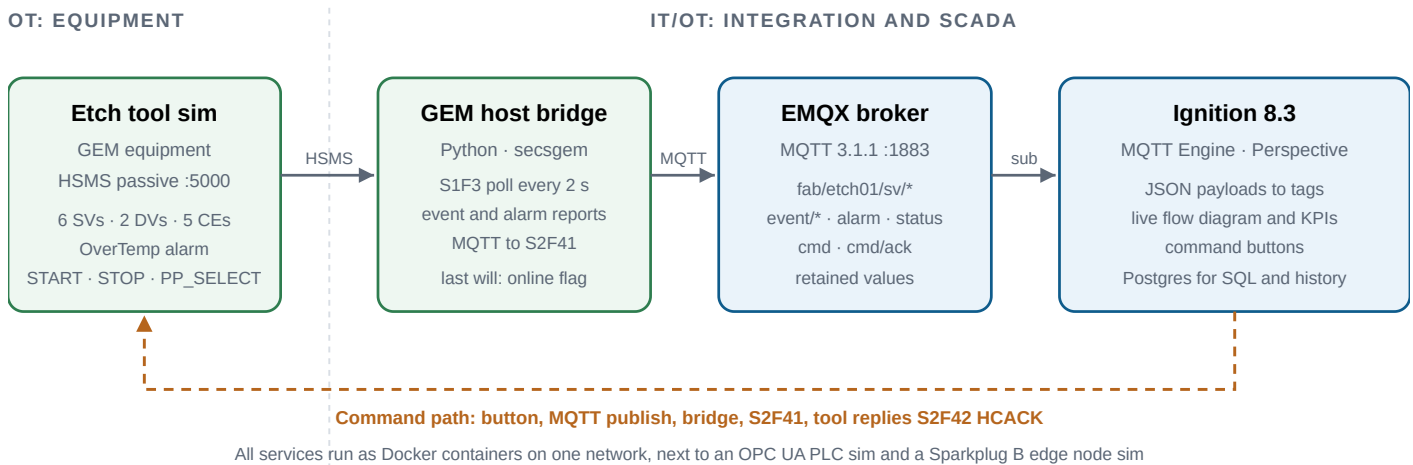
A closed-loop fab tool integration lab: equipment data up to SCADA, remote commands back down

Christoph Viehoff · Automation and Controls Engineer · Portland, OR

Home lab build, September 2026

This lab connects a simulated semiconductor etch tool to an Ignition 8.3 SCADA gateway using the same protocols a fab uses. The tool speaks SECS/GEM over HSMS. A Python GEM host collects its data and republishes it over MQTT, and Ignition turns it into live tags and an operator screen. Commands from the screen travel back down the same path and reach the tool as standard S2F41 remote commands.

Architecture



What it demonstrates

Equipment integration

- SECS/GEM host behavior: establish communication, subscribe to collection events, enable alarms, poll status variables
- Event reports carry context data (lot and wafer ID) with each WaferStart and WaferEnd
- Remote commands with parameters, acknowledged with HBACK

IT/OT architecture

- A unified namespace pattern: one topic tree per tool under `fab/<tool>`
- Decoupled layers: the tool, bridge, broker and SCADA can each be swapped or scaled on their own
- Bridge health through an MQTT last will, shown live on the screen

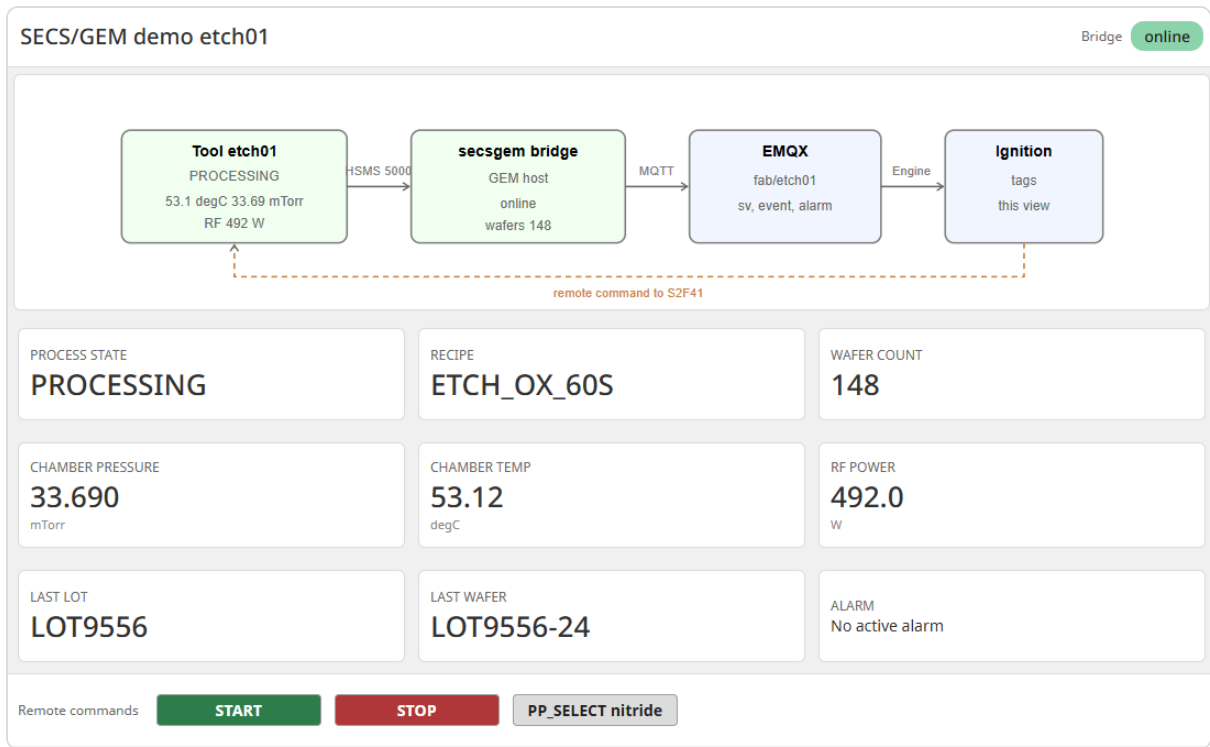
SCADA and HMI

- Ignition Perspective view with a live SVG flow diagram built from tag values
- Tiles for state, recipe, wafer count, chamber pressure, temperature and RF power
- Alarm tile that shows only active alarms

Engineering practice

- Everything defined as code: Docker Compose, Python, view JSON
- Repeatable import and backup scripts in PowerShell
- Issues traced end to end with logs and broker tools, documented on page 3

The operator screen



Live Ignition Perspective view. The flow diagram colors and values come from tags: the tool box turns green while PROCESSING, and the bridge box follows its online status. The buttons send remote commands to the tool.

A command round trip

- 1 The operator presses **PP_SELECT nitride**. A Perspective script publishes `{"rcmd": "PP_SELECT", "params": {"PPID": "ETCH_NIT_90S"}}}` to `fab/etch01/cmd` through MQTT Engine.
- 2 EMQX delivers it to the bridge, which is subscribed to the command topic.
- 3 The bridge sends **S2F41** Host Command Send to the tool over HSMS.
- 4 The tool replies **S2F42** with HCAK 4 (accepted, will finish later) and changes its recipe.
- 5 The bridge publishes the ack to `cmd/ack`. On the next S1F3 poll the new recipe shows on screen, within about 2 seconds.

Data model

Type	IDs	Items
SV	2101 to 2106	ChamberPressure, ChamberTemp, RFPower, ProcessState, WaferCount, Recipe
DV	2001, 2002	LotID, WaferID
CE	3001 to 3003	WaferStart, WaferEnd, RecipeSelected
CE	3101, 3102	OverTemp set and clear
ALID	5001	OverTemp, above 80 degC
RCMD		START, STOP, PP_SELECT(PPID)

MQTT topics under `fab/etch01`

<code>sv/<Name></code>	retained, <code>{"ts", "value"}</code>
<code>event/<Name></code>	CE with lot and wafer data
<code>alarm</code>	retained, set flag and text
<code>status</code>	online flag, last will
<code>cmd</code> , <code>cmd/ack</code>	remote command and HCAK

Build and debug notes

Getting the loop working end to end took five fixes. Each was found by checking one layer at a time: container network, broker, tag browser, binding preview, gateway console and bridge log.

Symptom	Cause	Fix
Bridge could not reach the broker	The add-on services joined Compose's default network, not the lab network that EMQX and Ignition use	Put every service on the shared <code>lab</code> network and made the bridge wait for EMQX to report healthy
Flow diagram showed ERROR, bridge showed null	The status payload is <code>{"online":true}</code> , so the tag is <code>status/online</code> . The view pointed at <code>status/online/value</code> , and one bad reference made the whole expression bad quality	Corrected the path in all four bindings
Alarm text stayed on after clearing	The retained alarm message carries text on both set and clear	Bound the tile to the <code>set</code> flag as well
Buttons said "sent" but the tool did nothing	The script published to a server named <code>emqx</code> , but MQTT Engine calls the connection <code>Chariot SCADA</code> . No error was raised	Moved the server name into one view property. Confirmed with the bridge log and a <code>mosquitto_sub</code> listener
Silent publish failures	The original scripts had no error handling	Byte payloads, try and except, and logging to the console and gateway log

Technology

SCADA	Ignition 8.3, Perspective, MQTT Engine
Equipment protocol	SECS-II, GEM, HSMS via Python secsgem 0.3
Messaging	EMQX, paho-mqtt, Sparkplug B sim
Other sources	OPC UA PLC simulator
Data	PostgreSQL 17
Platform	Docker Compose, PowerShell

How it scales to a fab

- One bridge per tool, or Ignition's own SECS/GEM module as host. A second sim tool on port 5001 is already set up for that
- Topic tree by site, area and tool, feeding MES, SPC and a historian from one broker
- Tag history on the process values for trends and FDC style limits
- TLS and per-client ACLs on the broker, command topics limited to approved roles
- Command audit trail from `cmd` and `cmd/ack` into SQL

One Definition, Every Pump

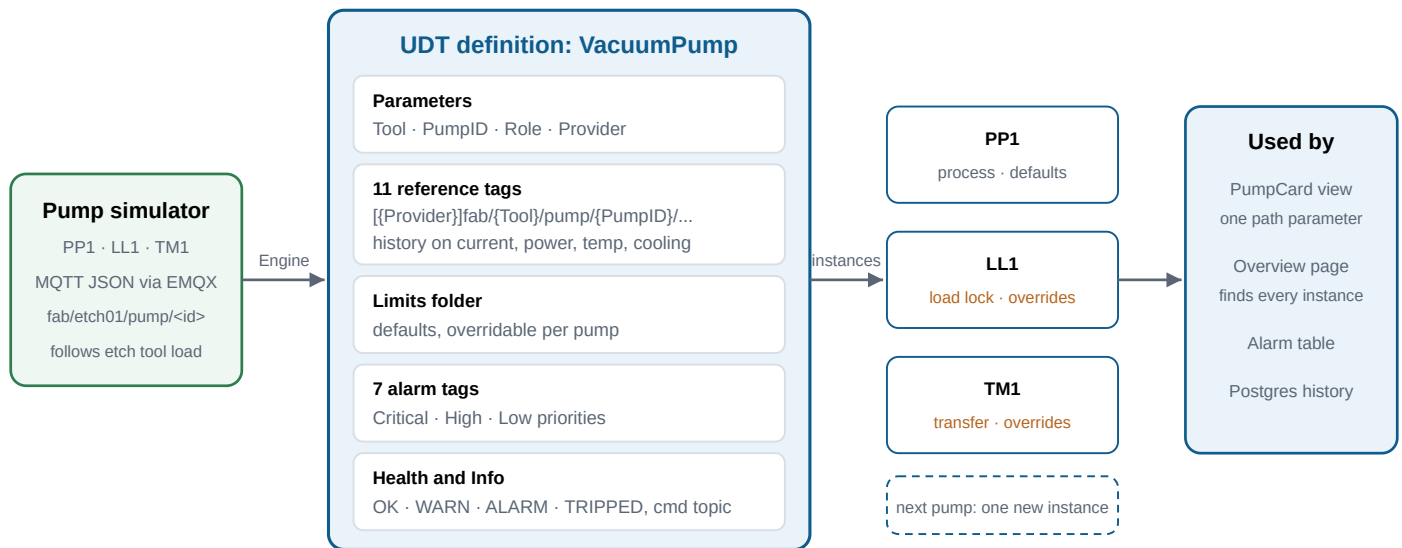
A VacuumPump UDT in Ignition 8.3: model once, deploy to any number of dry pumps

Christoph Viehoff · Automation and Controls Engineer · Portland, OR

Home lab build, September 2026

Fabs run hundreds of dry vacuum pumps, and every one needs the same data, limits, alarms, history and screen. This lab models the pump once as an Ignition User Defined Type (UDT). Each physical pump then becomes one instance with four parameters. Tags, alarms, history, health logic and the operator card all come from the definition. A change made in the definition reaches every pump at once.

How it fits together



Why a UDT

Consistency

- Every pump has the same tag names, units, formats and alarm set
- Screens, reports and analytics can rely on one structure
- No hand-built tags to drift apart over time

Change once, apply everywhere

- Edit a limit, alarm or expression in the definition and every instance updates at once
- Instances keep only their real differences as overrides, shown in bold

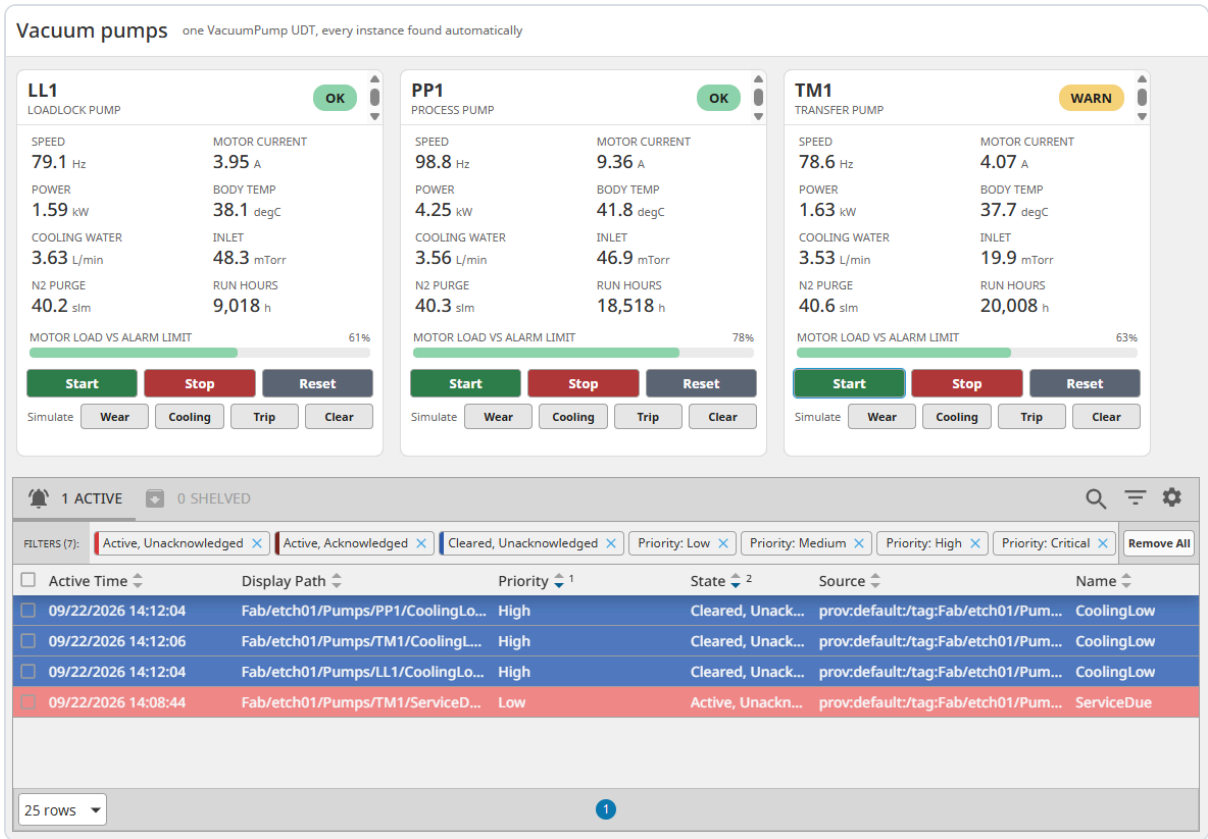
Fast to scale

- A new pump is one instance with four parameters
- The overview page browses for VacuumPump instances, so the new card appears with no screen work

Built-in intelligence

- Health state, load percentage and service due are computed in the tag layer, not in screens
- Alarms carry the pump's path, so operators see which pump at a glance

The result on screen



Ignition Perspective overview. Three cards from one PumpCard view, each bound to a different instance. TM1 shows WARN because it passed its 20,000 hour service interval, and the ServiceDue alarm is active. The blue rows are cleared start-up alarms, the timing case described on page 3.

Anatomy of the definition

Group	Members	Purpose
Parameters	Tool, PumpID, Role, Provider	The only per-pump inputs. They build every data path, the command topic and the labels.
Reference tags	State, Running, Speed, MotorCurrent, MotorPower, InletPressure, ExhaustPressure, BodyTemp, N2Purge, CoolingFlow, RunHours	Point at <code>[{Provider}]fab/{Tool}/pump/{PumpID}/<Metric>/value</code> . Units and formats set once. History to Postgres on the four trend signals.
Limits	CurrentWarn, CurrentAlarm, TempWarn, TempAlarm, MinCoolingFlow, ServiceHours	Memory tags with defaults. Smaller pumps override the current limits.
Alarm tags	Tripped, CurrentHighHigh, TempHighHigh, CoolingLow, CurrentHigh, TempHigh, ServiceDue	Boolean expressions against the limits, each with one alarm. Critical, High and Low priorities.
Derived	Health, CurrentLoadPct	One status word for screens and reports, and motor load as a percentage of the alarm limit.
Info	Tool, PumpID, Role, CmdTopic	Parameter values exposed as tags, so screens and scripts need only the instance path.

Demo scenarios

Scenario	What happens	What the UDT does
Bearing wear, PP1	Motor current climbs about 1.2 A per minute until the pump trips near 13.5 A	WARN at 10.5 A, High alarm at 12 A, Critical trip alarm. Load bar goes yellow, then red.
Loss of cooling, LL1	Cooling water drops to 0.9 L/min and body temperature climbs	CoolingLow High alarm at once, then TempHigh as the pump heats up
Service due, TM1	Run hours pass 20,000	Low priority ServiceDue alarm and WARN health, a hook for a maintenance work order
Process load, PP1	The etch tool starts PROCESSING	Current and power rise with gas load, showing the pump reacts to the tool it serves
Change for all	Lower TempWarn in the definition	All three pumps use the new limit at once, except where an instance has its own override

Adding a pump

- 1 In the Tags tab, right-click the Pumps folder and create a VacuumPump instance, for example PP2.
- 2 Set Tool and PumpID. Role and Provider keep their defaults.
- 3 Done. Tags, alarms, history and the overview card all appear. No screen changes.

Lessons from the build

- **Silent import loss.** Ignition 8.3 dropped the alarm list during import because of one unsupported field, with no error shown. I exported a hand-made alarm to get the gateway's exact format, then rebuilt the file from it.
- **Start-up race.** Running and CoolingFlow arrive as separate MQTT messages, so CoolingLow flickered on each start. The fix is a 5 second active delay on that alarm, set once in the definition for every pump.

How it scales to a fab

- One definition for each pump model, or a base type with child types for booster and dry stages
- Sub-fab monitoring for hundreds of pumps from one overview, filtered by tool or area
- Motor current and temperature history feeding predictive maintenance: rising current is an early sign of bearing wear
- ServiceDue and trip alarms routed to a CMMS as work orders
- The same pattern for chillers, abatement units, RF generators and gas boxes

Technology

SCADA	Ignition 8.3, UDTs, Perspective, alarming
Data path	Python sim, EMQX, MQTT Engine
History	PostgreSQL 17
Deployment	Docker Compose, JSON tag and view import